



A COMPLETE BEGINNER-TO-PRO LESSON

From Sand to Silicon

Semiconductors & chip design — ASIC, SoC, FPGA and GPU, Verilog, assembly and C, careers and a 12-month plan to start from where you are.

Verilog

RISC-V

FPGA

C

ASIC

SoC

Prepared for Samuel Nathaniel · September 2026

What you'll learn

0	Start here	2
	What this lesson covers, and why you are closer than you think	
1	The world of semiconductors	4
	From sand to the phone in your hand	
2	Digital logic	6
	The language of 0s and 1s	
3	How computers work inside	8
	CPUs, instruction sets, pipelines, memory and GPUs	
4	Types of chips: ASIC, SoC, FPGA, GPU...	10
	Choosing the right kind of silicon	
5	The chip design flow	12
	How an idea becomes a manufactured chip	
6	Verilog: describing hardware	14
	Your first hardware description language	
7	Verification: proving it works	17
	Testbenches, waveforms, UVM, cocotb and formal	
8	FPGAs in depth	19
	Re-programmable hardware you can own this month	
9	Assembly language	21
	Talking to the processor in its own words	
10	C for hardware	23
	Firmware, registers and bit manipulation	
11	Physical design, analog and manufacturing	25
	What happens after the Verilog	
12	Tools of the trade	26
	Commercial EDA and the open-source silicon movement	
13	Careers and opportunities	27
	Where the jobs are, and where you can fit in	
14	Your 12-month roadmap	29
	Starting from exactly where you are now	
15	Resources, glossary and quiz	31
	Everything to keep learning	

0

CHAPTER 0

Start here

What this lesson covers, and why you are closer than you think

Every phone, laptop, car, satellite and AI data centre runs on chips. Chip design is the craft of turning an idea — “add two numbers”, “recognise a face”, “send a 5G signal” — into billions of microscopic switches etched into silicon. This lesson takes you from **what a transistor is** all the way to **writing Verilog, reading assembly, writing C for hardware, running a real chip design flow**, and a **12-month plan** to get you there.

Understand

Semiconductors, logic, CPUs, and the difference between ASIC, SoC, FPGA and GPU.

Build

Verilog modules, testbenches, RISC-V assembly and embedded C — all tested examples.

Go pro

The industry map, the jobs, the tools, and exactly how to start this week.

■ You already know more than you think

You have shipped a real game — Orbit Rush — in JavaScript with a Java Android host, tests and Git. Many of those ideas map straight onto hardware:

What you did in Orbit Rush	The hardware version
<code>tick(dt)</code> updates the whole world 60 times per second	A clock updates every register in a chip, often a billion times per second
Screens: <code>home</code> → <code>playing</code> → <code>pause</code> → <code>result</code>	A finite-state machine (FSM) — the backbone of every controller circuit
<code>npm test</code> — 23 engine tests that must pass	A testbench that simulates the chip and checks every answer
Collision checks, score maths, bit-level save checksums	Combinational logic : adders, comparators, hash circuits
Java for Android, JavaScript for the game	C for firmware, Verilog for hardware, Python for verification
Git, reviews, regression tests	Exactly the same — chip teams live in Git and regression suites

◆ The one big mind-shift

In software, lines run **one after another**. In hardware, **everything runs at the same time**, all the time. A Verilog file is not a list of instructions — it is a **description of a circuit**. Once that clicks, the rest is practice.

■ How to use this lesson

- ✓ **Read one chapter a day.** Each ends with something to try.
- ✓ **Type the code yourself.** Every Verilog example here was simulated with Icarus Verilog; every C and assembly example was compiled with clang for RISC-V. They work.
- ✓ **Use the free browser tools first** (HDLBits, EDA Playground) — no installs, low data, works on any laptop.
- ✓ **Jump to Chapter 14** any time you want the step-by-step plan.

i About “FGPU”

You asked about “FGPU”. That is usually a mix of two words — **FPGA** (re-programmable hardware) and **GPU** (graphics / parallel processor). Both are covered in full: GPU in Chapters 3–4, FPGA in Chapter 8.

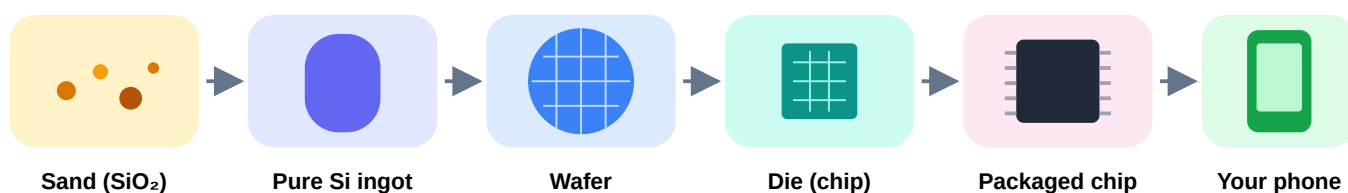
1

CHAPTER 1

The world of semiconductors

From sand to the phone in your hand

A **semiconductor** is a material whose ability to carry electricity sits between a conductor (copper) and an insulator (glass) — and, crucially, can be **controlled**. Silicon is the star: it is made from ordinary sand, and we can switch its conductivity on and off billions of times per second.



■ Doping: making silicon useful

N-type silicon

Add a pinch of **phosphorus** or arsenic. Extra **electrons** (negative) are free to move.

P-type silicon

Add a pinch of **boron**. It creates “**holes**” — missing electrons that behave like positive charges.

Put N and P side by side and you get a **PN junction** (a diode): current flows one way but not the other. Arrange N and P regions cleverly under a thin insulator and a “gate” electrode and you get the most important invention of the 20th century...

■ The MOSFET: a switch with no moving parts

A **MOSFET** (metal-oxide-semiconductor field-effect transistor) has three main terminals: **source**, **drain** and **gate**. The voltage on the gate decides whether current can flow from source to drain. That is it — a voltage-controlled switch.

NMOS

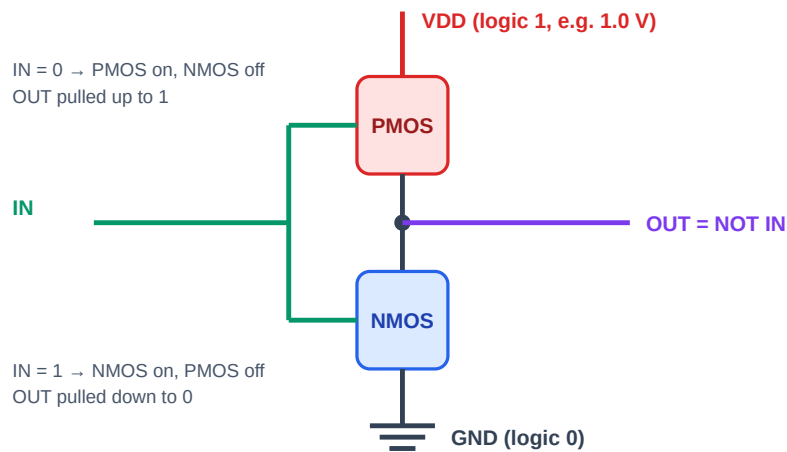
Turns **ON** when the gate is **high (1)**. Good at pulling a wire down to 0.

PMOS

Turns **ON** when the gate is **low (0)**. Good at pulling a wire up to 1.

■ CMOS: pairing them up

Almost every digital chip uses **CMOS** (complementary MOS): each gate has PMOS transistors pulling up and NMOS transistors pulling down. Exactly one side is on at a time, so very little current flows when nothing is changing. Here is the simplest gate, the **inverter** (NOT):



★ Key idea

Put two NMOS in series and two PMOS in parallel and you get a **NAND** gate. From NAND alone you can build *every* other logic function — and therefore an entire computer. (Chapter 14 shows you a free course that makes you do exactly that.)

■ Moore's law and “nanometres”

In 1965 Gordon Moore observed that the number of transistors on a chip was doubling roughly every year (he later revised it to about every two years). That trend held for decades, which is why today's flagship chips hold **tens of billions** of transistors. Process names like “5 nm” or “3 nm” are now **marketing labels** for a technology generation — no single feature is exactly that size — but smaller nodes still mean more transistors, more speed per watt, and much higher design cost.

■ Who does what: the industry map

Fabless designers

Design chips, outsource manufacturing: NVIDIA, AMD, Qualcomm, Apple, Broadcom, MediaTek.

Foundries

Manufacture other people's designs: TSMC, Samsung Foundry, Intel Foundry, GlobalFoundries, UMC, SMIC.

IDMs

Design *and* make their own chips: Intel, Samsung, Texas Instruments, Infineon, STMicro, NXP.

Equipment makers

Build the machines fabs use: ASML (lithography), Applied Materials, Lam Research, Tokyo Electron, KLA.

EDA software

The design tools: Synopsys, Cadence, Siemens EDA — plus a growing open-source world (Ch. 12).

IP & packaging

Licensed blocks (Arm, RISC-V cores) and assembly/test houses (ASE, Amkor, JCET).

i Scale

Global semiconductor sales are more than half a trillion US dollars a year, and the industry is investing heavily in new fabs across the US, Europe, Japan, India and elsewhere.

2

CHAPTER 2

Digital logic

The language of 0s and 1s

Everything a digital chip does comes down to wires carrying **0** (low voltage) or **1** (high voltage), and gates that combine them.

Number systems you will use every day

Decimal	Binary	Hex	Verilog literal
5	0101	5	4'b0101 or 4'd5
10	1010	A	4'b1010 or 4'hA
255	1111 1111	FF	8'hFF

In Verilog, 4'b0101 means “4 bits wide, binary, value 0101”. The width matters — hardware wires have a fixed number of bits.

The logic gates

Gate	Symbol in code	Output is 1 when...	A=0 B=0	0 1	1 0	1 1
AND	a & b	both inputs are 1	0	0	0	1
OR	a b	at least one input is 1	0	1	1	1
XOR	a ^ b	inputs are different	0	1	1	0
NAND	~(a & b)	not both 1	1	1	1	0
NOR	~(a b)	both are 0	1	0	0	0
NOT	~a	input is 0 (one input only)	0 → 1, 1 → 0			

Two families of circuits

Combinational

Output depends **only on the current inputs**. No memory. Examples: adders, multiplexers (selectors), comparators, decoders.

Sequential

Output depends on inputs **and stored state**. Uses **flip-flops** that capture a value on each clock edge. Examples: counters, registers, state machines, CPUs.

Flip-flops and the clock

A **D flip-flop** copies its input **D** to its output **Q** at the instant the clock rises (the **rising edge**) and holds it until the next edge. Group 32 flip-flops and you have a 32-bit **register**. The **clock** is a square wave that ticks through the whole chip — just like the game loop that ticks through Orbit Rush.

■ Timing: how fast can it go?

Signals take time to pass through gates. Between two flip-flops, the slowest chain of logic is the **critical path**. The clock period must be longer than that delay (plus a little setup time).

$$f_{\max} \approx 1 \div (\text{critical-path delay}) \rightarrow 2 \text{ ns path} \Rightarrow \text{about } 500 \text{ MHz}$$

- **Setup time** — the input must be stable a little *before* the clock edge.
- **Hold time** — and stay stable a little *after* it.
- Break those rules and the flip-flop can capture garbage. **Static timing analysis (STA)** tools check every path in the chip for you.

▶ Try it (15 minutes, free)

Play **nandgame.com** in your browser: you build gates, an adder, memory and finally a working computer from nothing but NAND gates. It is the most fun way to make this chapter stick.

3

CHAPTER 3

How computers work inside

CPUs, instruction sets, pipelines, memory and GPUs

A **CPU** is a sequential circuit that repeats three steps forever: **fetch** an instruction from memory, **decode** what it means, **execute** it. The list of instructions a CPU understands is its **instruction set architecture (ISA)**.

■ The main ISAs

ISA	Style	Where you find it	Licence
x86-64	CISC (many complex instructions)	Most PCs and servers (Intel, AMD)	Closed
Arm	RISC (simple, regular instructions)	Almost every phone, Apple Macs, many servers and microcontrollers	Licensed
RISC-V	RISC, modular	Microcontrollers, accelerators, research — growing fast	Open and free

★ Why RISC-V matters for you

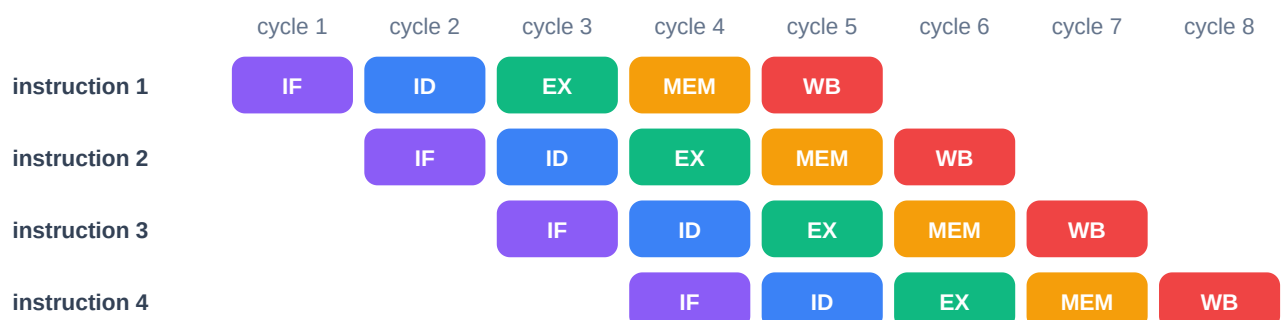
RISC-V is an **open standard**: anyone can design a RISC-V CPU without paying licence fees. That is why it is the best ISA to learn on, and why this lesson uses it for assembly.

■ Inside the CPU

- **Register file** — a few dozen very fast storage slots (RISC-V has 32).
- **ALU** (arithmetic logic unit) — adds, subtracts, ANDs, shifts, compares.
- **Control unit** — decodes each instruction and steers data to the right place.
- **Program counter (PC)** — the address of the next instruction.

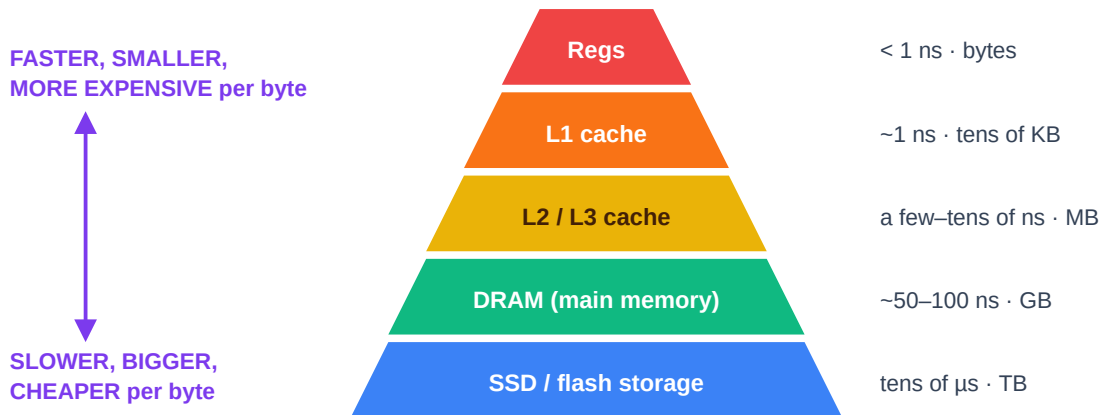
■ Pipelining: an assembly line for instructions

Instead of finishing one instruction before starting the next, a pipelined CPU overlaps them. The classic 5 stages are **IF** (fetch), **ID** (decode), **EX** (execute), **MEM** (memory access) and **WB** (write back):



Once full, the pipeline finishes one instruction **every cycle**. The hard parts — **hazards** — are when an instruction needs a result that isn't ready yet, or when a branch changes where to fetch from. Modern CPUs add branch prediction, out-of-order execution and several pipelines side by side.

■ The memory hierarchy



Fast memory is expensive and small; big memory is slow. Caches keep recently used data close to the CPU. A surprising amount of chip performance is really about **moving data**, not computing on it.

■ GPUs: thousands of small workers

A CPU has a few powerful cores optimised for one thread at a time. A **GPU** has **thousands of simpler cores** that run the same instruction on lots of data at once (SIMD / SIMT). That suits graphics (every pixel is independent) and the matrix maths behind AI — which is why GPUs power today's AI boom. Programmers reach GPUs through CUDA, OpenCL, Vulkan or Metal.

◆ Link to your game

Orbit Rush draws every star, rock and particle with the phone's GPU through Canvas 2D. Each frame, the CPU runs your `tick()` logic; the GPU fills millions of pixels in parallel.

Types of chips: ASIC, SoC, FPGA, GPU...

Choosing the right kind of silicon

“Chip” covers very different things. The biggest decision in any hardware project is **which kind** to use.

ASIC — Application-Specific Integrated Circuit

Custom silicon built for **one job**: a Bitcoin miner, a Wi-Fi radio, Google’s TPU. Best speed and power, lowest cost per unit at high volume — but very expensive and slow to design, and you **cannot change it** after manufacture.

SoC — System on Chip

A whole computer on one die: CPU cores, GPU, AI engine, memory controllers, radios and I/O. Your phone’s main chip is an SoC. Most SoCs are themselves ASICs.

FPGA — Field-Programmable Gate Array

A chip full of **configurable logic** you can rewire with a file in seconds. Slower and more power-hungry than an ASIC, but no factory needed. Used for prototyping, telecoms, finance, defence and low-volume products.

GPU — Graphics Processing Unit

A massively parallel processor (Chapter 3). Programmed with software, not rewired. Dominant for AI training.

CPU and MCU

General-purpose processors. A **microcontroller (MCU)** is a small, cheap CPU with memory and I/O built in — the brain of a washing machine, drone or smart meter.

And more

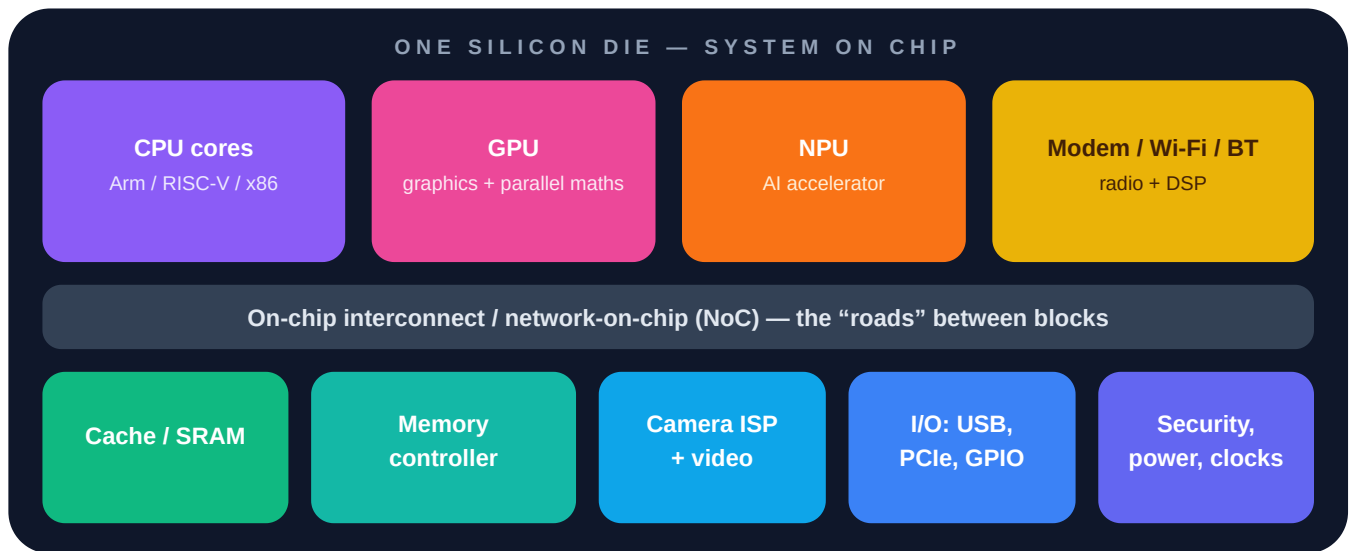
NPU/TPU (AI accelerators), **memory** (DRAM, flash), **analog & RF** (radios, sensors), **power** chips (chargers, motor drivers).

ASIC vs FPGA vs GPU vs MCU

	ASIC / SoC	FPGA	GPU	MCU
Change after build?	No	Yes — reload	Yes — software	Yes — firmware
Up-front cost	Very high (millions+)	Low	Low	Very low
Cost per unit	Lowest at volume	Medium-high	High	Very low
Speed & energy	Best	Good	Great for parallel maths	Modest
Time to first product	1–3 years	Weeks–months	Days	Days

Language	Verilog / SystemVerilog / VHDL	Verilog / VHDL (or HLS)	CUDA, OpenCL, Python	C / C++ / Rust
-----------------	--------------------------------	-------------------------	----------------------	----------------

■ What is inside a phone SoC



★ Rule of thumb

Start with **software** (CPU/MCU/GPU). Move to an **FPGA** when software is too slow or you need exact timing. Build an **ASIC** only when volume is huge or power/performance targets are impossible otherwise. Many ASICs are first prototyped on FPGAs.

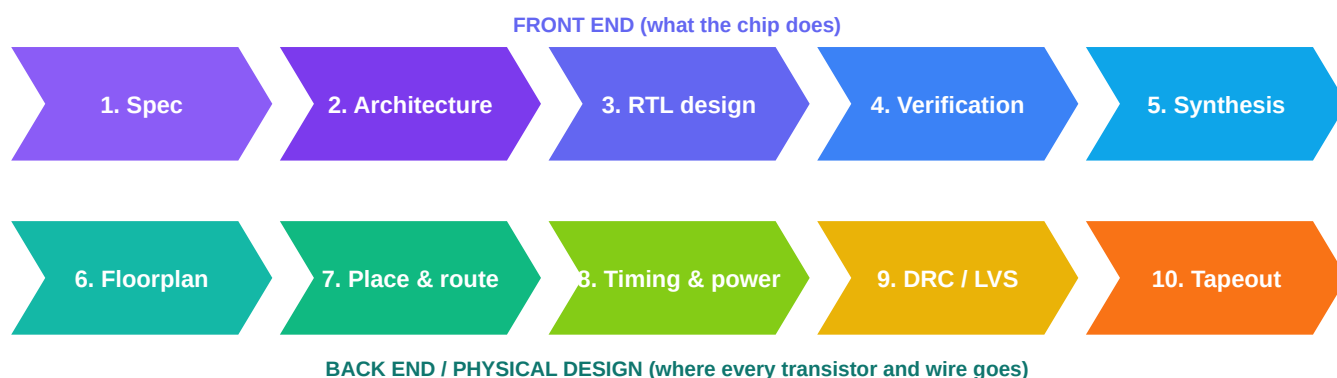
5

CHAPTER 5

The chip design flow

How an idea becomes a manufactured chip

A modern chip passes through ten broad stages. Big companies have separate teams for each; on small projects one person may do several.



Stage	What happens	Typical tools
1. Specification	What must the chip do? Speed, power, area, cost, interfaces.	Documents, spreadsheets
2. Architecture	Split into blocks, choose algorithms, model performance.	C++/Python/SystemC models
3. RTL design	Describe each block at register-transfer level in Verilog/VHDL.	Editor, linters (Verilator)
4. Verification	Prove the RTL is correct with simulation, random tests, formal proofs. Often the biggest effort.	Simulators, UVM, cocotb, formal tools
5. Synthesis	Convert RTL into a netlist of real gates from a cell library.	Synopsys Design Compiler, Cadence Genus, Yosys
6. Floorplanning	Decide where big blocks, power grid and I/O pins go.	Cadence Innovus, Synopsys ICC2, OpenROAD
7. Place & route	Place millions of cells; build the clock tree; connect every wire.	Same as above
8. Timing & power sign-off	Check every path meets timing at every corner; estimate power, IR drop.	PrimeTime, Tempus, OpenSTA
9. Physical verification	DRC: layout obeys the fab's rules. LVS: layout matches the netlist.	Calibre, Pegasus, Magic, KLayout, Netgen
10. Tapeout → fab → test	Send the final layout (GDSII/OASIS) to the foundry; chips come back months later for testing.	ATE testers, lab equipment

■ PPA: the three-way balancing act

Performance

How fast? Clock frequency, throughput.

Power

How much energy? Battery life, heat.

Area

How much silicon? Area is cost.

Improving one usually hurts another. Much of a chip designer's job is finding the best trade-off.

! Why mistakes are so expensive

Software bugs can be patched overnight. A bug found after tapeout can mean a **“respin”**: new masks, months of delay, and at leading-edge nodes costs that can run into many millions of dollars. That is why verification gets so much attention — and why verification engineers are always in demand.

6

CHAPTER 6

Verilog: describing hardware

Your first hardware description language

Verilog (and its modern superset **SystemVerilog**) is the most widely used hardware description language (HDL). It looks a little like C, but remember: **you are drawing a circuit, not writing a program.**

1. A module is a box with pins

This full adder adds three bits. `assign` creates **continuous** logic: the output always reflects the inputs, just like real wires and gates.

gates.v

VERILOG

```
// A 1-bit full adder built from gates
module full_adder (
    input wire a, b, cin,
    output wire sum, cout
);
    assign sum = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule
```

Simulation result

full_adder errors=0 (all 8 input combinations checked)

2. Sequential logic: a counter

`always @(posedge clk)` means “on every rising clock edge”. Everything inside becomes flip-flops.

counter.v

VERILOG

```
// 4-bit counter with synchronous reset and enable
module counter4 (
    input wire clk,
    input wire rst, // 1 = go back to zero
    input wire en, // 1 = count
    output reg [3:0] count
);
    always @(posedge clk) begin
        if (rst) count <= 4'd0;
        else if (en) count <= count + 4'd1;
    end
endmodule
```

Piece

Meaning

`reg [3:0] count`

A 4-bit value that is stored (bits 3 down to 0)

<code><=</code>	Non-blocking assignment: all flip-flops update together at the clock edge. Use it in clocked blocks.
<code>4'd1</code>	4-bit decimal constant 1
<code>if (rst) ... else if (en)</code>	Becomes a multiplexer in front of the flip-flops

■ 3. A finite-state machine: traffic light

FSMs are everywhere: protocol controllers, CPUs, game logic. This one is green for 9 cycles, amber for 3, red for 7:

```

traffic.v
VERILOG

// Traffic light: a finite-state machine (FSM)
module traffic_light (
    input wire clk, rst,
    output reg [2:0] light // {red, amber, green}
);
    localparam GREEN = 2'd0, AMBER = 2'd1, RED = 2'd2;
    reg [1:0] state;
    reg [3:0] timer;

    always @(posedge clk) begin
        if (rst) begin
            state <= RED; timer <= 4'd0;
        end else if (timer == 4'd0) begin
            case (state) // move to the next colour
                GREEN: begin state <= AMBER; timer <= 4'd2; end
                AMBER: begin state <= RED; timer <= 4'd6; end
                default: begin state <= GREEN; timer <= 4'd8; end
            endcase
        end else begin
            timer <= timer - 4'd1; // count down
        end
    end

    always @(*) begin // outputs depend only on state
        case (state)
            GREEN: light = 3'b001;
            AMBER: light = 3'b010;
            default: light = 3'b100;
        endcase
    end
endmodule

```

Simulated output ({red, amber, green})

```

light each clock: 001 001 001 001 001 001 001 001 001 010 010 010 100 100 100 100 100 100
001 ...
                \_____ green (9) _____/ \_amber_/ \_____ red (7) _____/ green again

```


■ Blocking vs non-blocking — the classic trap

`<=` non-blocking

Use in `always @(posedge clk)`. Every right-hand side is read first, then all registers update at once — exactly like real flip-flops.

`=` blocking

Use in `always @(*)` for combinational logic. Updates immediately, in order, like normal code.

! Common beginner mistakes

- **Accidental latches:** in an `always @(*)` block, forgetting an `else` or a `default:` makes the tool “remember” the old value — a latch you didn’t want.
- **Mixing `=` and `<=`** in the same block.
- **Two blocks driving the same signal** (multiple drivers).
- **Unsynthesisable code:** delays like `#5`, `$display` and `initial` work in simulation but **do not** become hardware.

■ The HDL family

SystemVerilog

Industry standard for design and verification. Learn after Verilog.

VHDL

Strongly typed; popular in Europe, aerospace and defence.

Chisel · SpinalHDL · Amaranth

Modern HDLs embedded in Scala or Python that generate Verilog.

HLS

High-level synthesis: write C/C++ and let a tool generate hardware (e.g. AMD Vitis HLS).

SystemC

C++ library for architecture and performance models.

Tcl

The scripting language nearly every EDA tool is driven with.

▶ Try it (free, in your browser)

Go to **hdlbits.01xz.net** and solve the first 20 problems. It checks your Verilog instantly. Then paste the counter above into **edaplayground.com**, pick Icarus Verilog, and run it with the testbench in the next chapter.

7

CHAPTER 7

Verification: proving it works

Testbenches, waveforms, UVM, cocotb and formal

On a chip project, **verification often takes more effort than design**. The goal: find every bug before the chip is manufactured.

■ A self-checking testbench

A testbench is Verilog that is **never built into hardware**. It creates a clock, drives inputs into the design under test (DUT) and checks the outputs.

tb_counter.v

VERILOG

```
// Testbench: drives the counter and checks it
`timescale 1ns/1ps
module tb_counter;
    reg clk = 0, rst = 1, en = 0;
    wire [3:0] count;
    counter4 dut (.clk(clk), .rst(rst), .en(en), .count(count));

    always #5 clk = ~clk;                // 100 MHz clock (10 ns period)

    initial begin
        $dumpfile("counter.vcd");        // waveform for GTKWave
        $dumpvars(0, tb_counter);
        #12 rst = 0; en = 1;             // release reset, start counting
        repeat (5) @(posedge clk);
        #1;
        if (count !== 4'd5) $display("FAIL: count = %0d", count);
        else $display("PASS: count = %0d", count);
        $finish;
    end
endmodule
```

Run: `iverilog -o sim counter.v tb_counter.v && vvp sim`

```
PASS: count = 5
tb_counter.v:18: $finish called at 56000 (1ps)
```

Open `counter.vcd` in **GTKWave** or **Surfer** to see every signal as a waveform — the hardware engineer's debugger.

■ The verification toolbox

Technique	What it does
Directed tests	Hand-written cases like the one above. Good for bring-up.
Constrained-random	Generate thousands of random but legal inputs to find cases nobody thought of.

Scoreboards & reference models	Compare the DUT against a simple “golden” model (often in C or Python).
Coverage	Measure which states, values and combinations have been tested. “Done” means coverage goals are met.
Assertions (SVA)	Rules checked continuously, e.g. “grant never comes without request”.
UVM	The industry-standard SystemVerilog framework for reusable testbenches. Heavily used in job descriptions.
Formal verification	Mathematically proves properties for <i>all</i> inputs. Open-source: SymbiYosys.
Emulation / FPGA prototyping	Run the design on huge FPGA systems to boot real software before tapeout.

■ cocotb: verification in Python

If you like writing tests in JavaScript, you’ll like **cocotb**: write testbenches in Python and drive any simulator.

```

test_counter.py

import cocotb
from cocotb.clock import Clock
from cocotb.triggers import RisingEdge

@cocotb.test()
async def counts_up(dut):
    cocotb.start_soon(Clock(dut.clk, 10, units="ns").start())
    dut.rst.value = 1
    dut.en.value = 0
    await RisingEdge(dut.clk)
    dut.rst.value = 0
    dut.en.value = 1
    for _ in range(5):
        await RisingEdge(dut.clk)
    assert dut.count.value == 5

```

◆ Your advantage

You already think in tests: Orbit Rush has engine tests and browser tests, and every bug fix came with a regression test that failed first. That is **exactly** the verification mindset. Python + cocotb is a realistic first step into a verification career.

8

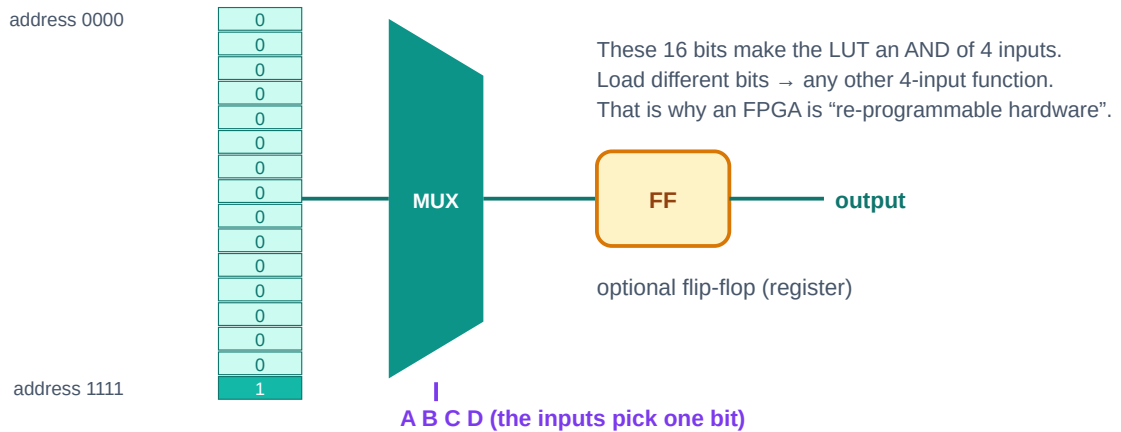
CHAPTER 8

FPGAs in depth

Re-programmable hardware you can own this month

An FPGA is a grid of small configurable blocks connected by programmable wires. You write Verilog, a tool turns it into a **bitstream**, and loading that bitstream turns the FPGA into your circuit.

A 4-input LUT = a tiny 16-bit memory + a selector



What is inside

LUTs

Look-up tables that implement any small logic function.

Flip-flops

Registers next to every LUT.

Block RAM

On-chip memories for buffers, FIFOs, frame buffers.

DSP slices

Hard multipliers for filters, audio, AI maths.

Clocking (PLLs)

Generate the exact clock frequencies you need.

I/O & transceivers

Pins for LEDs, VGA, HDMI, Ethernet, fast serial links.

The FPGA flow



Plus a **constraints file** that says which physical pin each signal uses and what clock speed you need.

Vendors

AMD (formerly Xilinx) and **Altera** (formerly part of Intel) lead the high end; **Lattice**, **Microchip**, **Gowin** and **Efinix** make smaller, cheaper, lower-power parts. Some FPGAs include hard Arm CPUs — an “FPGA SoC” (e.g. AMD Zynq).

■ Beginner boards

Board	FPGA	Tools	Why pick it	Rough price*
Sipeed Tang Nano 9K / 20K	Gowin	Gowin IDE or open-source (Yosys + nextpnr)	Cheapest real option; HDMI output	≈ US\$15-40
iCEBreaker / iCESugar	Lattice iCE40	Fully open-source	Great learning community; small and simple	≈ US\$40-80
Digilent Basys 3	AMD Artix-7	AMD Vivado (free edition)	University standard; switches, LEDs, VGA	≈ US\$150+
Terasic DE10-Lite	Altera MAX 10	Quartus Prime Lite (free)	Common in courses; lots of I/O	≈ US\$80-150

*Prices change often and differ by country and shipping — check current listings before you buy.

◆ A dream first project: Orbit Rush in pure hardware

Once you can blink an LED, build **Pong on a VGA or HDMI screen** — no CPU, no software, just Verilog: a counter for the screen position, registers for the ball and paddles, and comparators for collisions. Then grow it into a tiny Orbit Rush: a ship, falling rocks and a score. It is a brilliant portfolio piece.

9

CHAPTER 9

Assembly language

Talking to the processor in its own words

Assembly is a human-readable form of the machine instructions a CPU actually executes. You rarely write large programs in it, but understanding it shows you exactly what hardware does — essential for CPU design, firmware, debugging and performance work.

■ RISC-V basics

Registers

32 registers: `x0 – x31`, with friendly names. `zero` is always 0; `a0 – a7` hold arguments and return values; `t0 – t6` are temporaries; `sp` is the stack pointer; `ra` the return address.

Instruction groups

Arithmetic: `add`, `sub`, `addi`

Logic: `and`, `or`, `xor`, `slli`

Memory: `lw` (load), `sw` (store)

Control: `beq`, `bne`, `blt`, `jal`

■ Example: add up 1 to 10

```

sum.s
# sum = 1 + 2 + ... + 10, result in a0
.globl main
main:
    li    t0, 1      # i = 1
    li    t1, 10     # limit = 10
    li    a0, 0      # sum = 0
loop:
    add    a0, a0, t0 # sum = sum + i
    addi   t0, t0, 1  # i = i + 1
    ble    t0, t1, loop # if i <= limit, go to loop
    ret                     # return sum (55) in a0

```

Step by step: `t0` is the counter *i*, `t1` the limit, `a0` the running total. Each pass adds *i* to the total, increments *i*, and jumps back while $i \leq 10$. The answer, 55, is returned in `a0`.

■ From assembly to machine code

The assembler turns each line into a 32-bit number. When clang assembled the example, `add a0, a0, t0` became:

`add a0, a0, t0` → 0x00550533 → 00000000 00101 01010 000 01010 0110011

Those bit fields say “R-type ALU operation, add, source registers `t0` (x5) and `a0` (x10), destination `a0` (x10)”. **Your CPU’s decoder in Verilog reads exactly these bits.** Also notice `ble` is a *pseudo-instruction*: the assembler rewrote it as `bge t1, t0, loop`.

A surprise from the compiler

We compiled the same loop written in C with `clang -O1` for RISC-V. The compiler did not produce a loop at all: it rewrote the maths as a formula based on $n \times (n - 1)$ and emitted a multiply. Optimising compilers are clever, which is one reason reading assembly is so useful.

Try it

Use **Ripes** (a free visual RISC-V simulator that animates the 5-stage pipeline), **RARS**, or the web-based **Venus**. Type the program above, step through it and watch `a0` reach 55.

10

CHAPTER 10

C for hardware

Firmware, registers and bit manipulation

C is the language of firmware, drivers, operating systems and the software that brings new chips to life. Chip companies hire many engineers who write C for their silicon.

Memory-mapped I/O: controlling hardware by writing to addresses

On a microcontroller, hardware blocks appear as **registers at fixed memory addresses**. Writing a 1 to the right bit turns a pin on. This example blinks an LED on an imaginary chip:

blink.c

```
#include <stdint.h>

/* Imaginary microcontroller: a GPIO block at address 0x40020000 */
#define GPIO_BASE    0x40020000u
#define GPIO_DIR     (*(volatile uint32_t *) (GPIO_BASE + 0x00)) /* 1 = output */
#define GPIO_OUT     (*(volatile uint32_t *) (GPIO_BASE + 0x04)) /* pin levels */
#define LED_PIN      5

static void delay(volatile uint32_t n) { while (n--) { } }

int main(void) {
    GPIO_DIR |= (1u << LED_PIN);          /* set pin 5 as an output */
    for (;;) {
        GPIO_OUT ^= (1u << LED_PIN);      /* toggle the LED */
        delay(500000);                    /* wait a little */
    }
}
```

Concept	Why it matters
<code>volatile</code>	Tells the compiler the value can change outside the program (it is hardware), so it must really read/write every time.
<code>uint32_t</code>	Exact-width integer types from <code><stdint.h></code> — registers have exact sizes.
<code>(1u << LED_PIN)</code>	A bit mask with only bit 5 set.
<code> =</code> , <code>&=</code> , <code>~</code> , <code>^=</code>	Set, clear and toggle bits without disturbing the others.

The four bit tricks you will use forever

bits.c (excerpt)

```
reg |= (1u << n);          /* set bit n */
reg &= ~(1u << n);         /* clear bit n */
reg ^= (1u << n);          /* toggle bit n */
bit  = (reg >> n) & 1u;    /* read bit n */
```


Output of the full test program

```
set:8 clear:0 toggle:1 test:1
```

■ Other C and C++ skills for chip work

- **Pointers and memory layout** — structs that match register maps, alignment, endianness.
- **Interrupts** — hardware pausing your code to handle an event.
- **RTOS and embedded Linux** — FreeRTOS, Zephyr; Linux device drivers for SoCs.
- **C/C++ reference models** — the “golden” model a testbench compares against.
- **Python** for test automation and cocotb; **Tcl** and **Bash/Make** for driving EDA tools.

■ Beginner microcontroller boards

Raspberry Pi Pico

Very cheap, excellent C SDK and documentation; also runs MicroPython.

ESP32

Built-in Wi-Fi and Bluetooth — perfect for IoT projects. Newer versions (ESP32-C3/C6) use RISC-V cores.

STM32 (Nucleo, “Blue Pill”)

Arm Cortex-M; widely used in industry. Great for learning registers properly.

Arduino Uno

The friendliest start, huge community. Move to “bare-metal” C once comfortable.

Physical design, analog and manufacturing

What happens after the Verilog

Physical design (back end)

- **Standard cells** — pre-designed gates (NAND2, DFF, ...) of fixed height, placed in rows like bricks.
- **Placement** — position millions of cells to keep wires short and timing met.
- **Clock tree synthesis** — deliver the clock to every flip-flop at nearly the same instant.
- **Routing** — connect everything across 10–15+ metal layers.
- **Sign-off** — STA, power/IR-drop, DRC, LVS, antenna checks.

$$\text{Dynamic power} \approx \alpha \cdot C \cdot V^2 \cdot f \text{ (activity} \times \text{capacitance} \times \text{voltage}^2 \times \text{frequency)}$$

Because voltage is **squared**, lowering the supply voltage is the most powerful way to save energy — which is why chips use many voltages and switch off unused blocks.

Analog and mixed-signal

The real world is analog. Every chip needs blocks that are designed transistor by transistor: **amplifiers, ADCs and DACs** (analog↔digital), **PLLs** (clock generators), **voltage regulators**, **SerDes** (very fast serial links) and **RF** radios. Analog design is harder to learn and to automate — so skilled analog engineers are rare and highly valued.

Manufacturing in a fab



These steps repeat hundreds of times over roughly three months. The most advanced chips use **EUV lithography** with 13.5 nm light, in machines made only by ASML. Each chip design needs a set of **masks** — one per patterned layer.

Packaging and test

- **Packaging** connects the tiny die to the outside world. Advanced packaging stacks or places several dies together: **chiplets**, 2.5D (side-by-side on an interposer, e.g. GPUs with HBM memory) and 3D stacking.
- **Test** — **DFT** (design for test) adds scan chains and built-in self-test (BIST) so automatic test equipment (ATE) can check every chip quickly.
- **Yield** — the fraction of good chips per wafer. Smaller dies and mature designs give better yield.

12

CHAPTER 12

Tools of the trade

Commercial EDA and the open-source silicon movement

Task	Commercial (industry)	Free / open-source (start here)
Simulation	Synopsys VCS, Cadence Xcelium, Siemens Questa	Icarus Verilog, Verilator , cocotb
Waveforms	Verdi, SimVision	GTKWave, Surfer
Synthesis	Design Compiler / Fusion Compiler, Genus	Yosys
Place & route	Innovus, IC Compiler II	OpenROAD , wrapped by OpenLane / LibreLane
Timing	PrimeTime, Tempus	OpenSTA
Layout & DRC/LVS	Virtuoso, Calibre	Magic, KLayout, Netgen
Formal	JasperGold, VC Formal	SymbiYosys
FPGA	AMD Vivado, Quartus, Gowin, Lattice Radiant	Yosys + nextpnr
Online, no install	—	EDA Playground, HDLBits

■ Open PDKs: real manufacturing processes, free to use

A **PDK** (process design kit) describes a fab's process: cell libraries, rules and models. Several are now open-source:

SkyWater SKY130

130 nm; the process that kick-started open-source chip design.

GF180MCU

GlobalFoundries 180 nm; good for mixed-signal and higher voltages.

IHP SG13G2

130 nm SiGe BiCMOS from Germany; strong for RF.

★ You can get a real chip made

Tiny Tapeout (tinytapeout.com) combines hundreds of small designs from people all over the world onto one shared chip, making a real tapeout affordable for individuals and students. Beginners have designed their first ASIC this way, starting from the browser. Check the website for current shuttles, deadlines and prices.

! Commercial tool access

Commercial EDA licences are very expensive and usually only available through employers or universities. Don't let that stop you — the open-source flow teaches the same concepts, and hiring managers respect a working open-source tapeout or FPGA project.

■ The main roles

Role	What you do	Key skills
RTL / design engineer	Write the Verilog/SystemVerilog for chip blocks	Digital logic, SV, architecture, timing
Design verification (DV)	Build testbenches and find bugs before tapeout	SystemVerilog, UVM, Python, C++, debugging
Physical design	Floorplan, place & route, close timing	STA, Tcl, EDA tools, CMOS basics
DFT engineer	Make chips testable (scan, BIST, ATPG)	Test theory, EDA tools, scripting
Analog / mixed-signal	Design amplifiers, ADCs, PLLs, regulators	Circuit theory, SPICE, layout
FPGA engineer	Build products and prototypes on FPGAs	Verilog/VHDL, Vivado/Quartus, interfaces
Embedded / firmware	C software that runs on chips; bring-up and drivers	C, assembly, RTOS, Linux, debugging with hardware
Computer architect	Decide how the chip should be organised	Performance modelling, C++/Python, deep CPU/GPU knowledge
EDA / CAD software	Build and automate the design tools themselves	Software engineering, algorithms, Python/C++
Process, yield, test & product	Manufacturing, quality, customer applications	Physics, statistics, lab skills, communication

■ Where the industry is heading

AI accelerators

Every major company is building custom AI chips, from data centres to phones.

RISC-V

Open CPUs in microcontrollers, cars, storage and AI.

Chiplets & packaging

Building big systems from smaller dies.

Automotive & energy

EVs, solar inverters and power electronics need many chips.

Edge AI & IoT

Small, low-power chips for sensors, farms, health and cities.

Security

Hardware roots of trust, secure enclaves, cryptography engines.

■ Being honest about location

Most ASIC design, fab and physical-design jobs are concentrated in the US, Taiwan, South Korea, Japan, China, India, Israel and parts of Europe. Local chip-design companies are still rare in Nigeria and much of Africa. That doesn't close the door — it tells you **which doors to use**:

Remote-friendly paths

Design verification, FPGA work, embedded firmware, EDA/automation software and technical writing can often be done remotely or as contracts.

Build locally

IoT and embedded products for local problems: solar and battery controllers, smart metering, agritech sensors, medical devices. MCUs and FPGAs make this possible today.

Study abroad

Funded MSc and PhD positions in microelectronics exist in Europe, the US and Asia. A strong public portfolio makes applications far stronger.

Your software edge

Chip teams need software people: test automation, EDA scripting, simulation infrastructure, firmware. JavaScript and Java experience transfers well to Python and C.

★ What gets you hired

Not certificates alone — **evidence**. A GitHub profile with working Verilog, testbenches that pass, waveforms in the README, an FPGA demo video, and a Tiny Tapeout design will speak for you anywhere in the world.

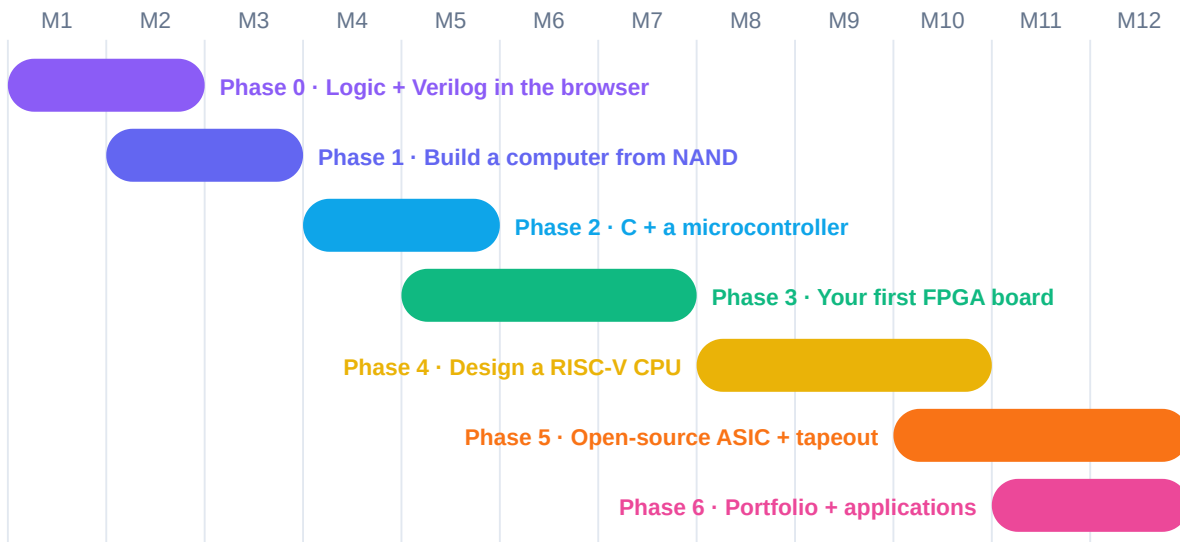
14

CHAPTER 14

Your 12-month roadmap

Starting from exactly where you are now

About **1-2 hours a day**. Phases overlap on purpose. Keep everything on GitHub from day one.



Phase 0 · Months 1–2 · Logic and Verilog in the browser

Cost: free

- Play **nandgame.com** to the end.
- Solve **HDLBits** problems daily (aim for 100+).
- Run the examples from Chapters 6–7 on **EDA Playground**; then install **Icarus Verilog** + **GTKWave** locally.
- Read *Digital Design and Computer Architecture (RISC-V Edition)* by Harris & Harris, chapters 1–4.

Phase 1 · Months 2–3 · Build a computer from NAND

Cost: free

- Work through **Nand2Tetris** (nand2tetris.org, also on Coursera) part 1: gates → ALU → CPU → assembler.
- Learn RISC-V assembly with **Ripes**; write small programs (sum, max of an array, string length).

Phase 2 · Months 4–5 · C and a microcontroller

Cost: ≈ US\$5–15

- Get a **Raspberry Pi Pico** or **ESP32**. Blink an LED, read a button, drive a display, use a timer interrupt.
- Write at least one driver directly with registers (no library).
- Learn Python basics and **cocotb**.

Phase 3 · Months 5–7 · Your first FPGA

Cost: ≈ US\$20–80

- Buy a **Tang Nano 9K** or **iCEBreaker**. Blink → 7-segment counter → **UART** (send text to your PC) → **VGA/HDMI Pong**.
- Grow Pong into a tiny **Orbit Rush in hardware**. Film it.

Phase 4 · Months 8–10 · Design your own RISC-V CPU

Cost: free

- Build a single-cycle RV32I core in Verilog, then pipeline it. Run your own assembly on it — on your FPGA.
- Verify it with a cocotb testbench and the official RISC-V architecture tests.

Phase 5 · Months 10–12 · Open-source ASIC

Cost: free to start; tapeout optional

- Take a small design (e.g. your Pong or a UART) through **OpenLane** with the SKY130 or IHP PDK: synthesis → layout → GDS.
- Submit it to a **Tiny Tapeout** shuttle if you can — then you have designed a real chip.

Phase 6 · Months 11–12+ · Portfolio and applications

- Polish 3–4 projects with clear READMEs, diagrams, waveforms and demo videos.
- Write short blog posts about what you learned. Join communities (r/FPGA, Tiny Tapeout Discord, FOSSi Foundation).
- Apply for internships, remote DV/FPGA/firmware roles, and funded postgraduate programmes.

■ Your first week, day by day

Day	Task	Time
1	Read Chapters 0–2 of this lesson. Start nandgame.com.	1.5 h
2	Create a free HDLBits account; solve the “Getting Started” and “Basics” sets.	1 h
3	Run <code>counter.v</code> + <code>tb_counter.v</code> on EDA Playground. Change it to count down.	1 h
4	Write the full adder, then a 4-bit adder made of four full adders. Test all 256 input pairs.	1.5 h
5	Modify the traffic light to add a pedestrian button input.	1 h
6	Create a GitHub repo “learning-verilog”; push everything with a README.	1 h
7	Rest, then answer the quiz at the end of this lesson.	30 min

! Tips for learning with limited power or data

Browser tools (HDLBits, nandgame, EDA Playground) use little data. Download PDFs and videos when you have good connectivity. Icarus Verilog and Yosys run well on modest laptops. Ask sellers about shipping times for boards and order early.

15

CHAPTER 15

Resources, glossary and quiz

Everything to keep learning

Books

Book	Best for
<i>Digital Design and Computer Architecture, RISC-V Edition</i> — Harris & Harris	The best single starting book: logic → Verilog → RISC-V CPU
<i>The Elements of Computing Systems</i> — Nisan & Schocken	The Nand2Tetris book
<i>Computer Organization and Design, RISC-V Edition</i> — Patterson & Hennessy	Computer architecture
<i>CMOS VLSI Design</i> — Weste & Harris	Transistor-level and physical design
<i>Design of Analog CMOS Integrated Circuits</i> — Razavi	Analog design
<i>The C Programming Language</i> — Kernighan & Ritchie	C, concisely

Free online

Practice

HDLBits · **EDA Playground** · **nandgame.com** · **Nand2Tetris** · **Ripes** RISC-V simulator

Courses & lectures

MIT OpenCourseWare 6.004 (Computation Structures) · **Onur Mutlu's** Digital Design & Computer Architecture lectures (ETH Zürich, YouTube) · **Ben Eater's** breadboard computer videos

Reading

ZipCPU blog (FPGA and formal) · **ChipVerify** (Verilog/SV/UVM tutorials) · RISC-V specifications (riscv.org)

Community

Tiny Tapeout · **FOSSi Foundation** · r/FPGA · r/chipdesign · local IEEE student branches

Glossary

ALU — circuit that does arithmetic and logic.

ASIC — chip custom-made for one application.

Bitstream — file that configures an FPGA.

Clock — square wave that times all registers.

CMOS — logic built from paired NMOS/PMOS transistors.

Critical path — slowest logic path; limits clock speed.

DFT — design for test; makes chips testable.

DRC / LVS — layout rule check / layout versus schematic.

DUT — design under test.

EDA — electronic design automation (chip design software).

Fabless — company that designs but doesn't manufacture.

Flip-flop — 1-bit memory updated on a clock edge.

Foundry — factory that makes chips for others.

FPGA — re-programmable logic chip.

FSM — finite-state machine.

GDSII — file format of the final chip layout.

HDL — hardware description language (Verilog, VHDL).

IP — reusable, licensable design block.

ISA — instruction set architecture.

LUT — look-up table, the FPGA's logic element.

MOSFET — the transistor used in digital chips.

Netlist — list of gates and their connections.

Node — manufacturing technology generation (e.g. "7 nm").

PDK — process design kit from a foundry.

PPA — performance, power, area.

RTL — register-transfer level description of logic.

SoC — system on chip.

STA — static timing analysis.

Synthesis — turning RTL into gates.

Tapeout — sending the final design to the fab.

Testbench — code that simulates and checks a design.

UVM — standard SystemVerilog verification framework.

Yield — fraction of working chips.

■ Quick quiz

1. Which transistor turns on when its gate is 0: NMOS or PMOS?
2. Which gate alone can build every other logic function?
3. What is the difference between combinational and sequential logic?
4. In a clocked `always` block, should you use `=` or `<=`?
5. A critical path is 4 ns long. Roughly what is the maximum clock frequency?
6. Name two reasons to choose an FPGA over an ASIC.
7. What does a 4-input LUT store, and how many bits?
8. In RISC-V, which register always reads as zero?
9. Why is `volatile` used for hardware registers in C?
10. What is Tiny Tapeout?

Answers: 1 PMOS · 2 NAND (or NOR) · 3 combinational has no memory; sequential stores state in flip-flops · 4 `<=` · 5 about 250 MHz · 6 reprogrammable, low up-front cost, fast time to market · 7 the truth table of its function — 16 bits · 8 `x0` / `zero` · 9 so the compiler performs every read and write, because hardware can change the value · 10 a shared, low-cost way for individuals to get a small design manufactured on a real chip.

■ You can do this.

Every chip designer started with a blinking LED and a confusing waveform. You have already built and shipped a real game — the same curiosity, patience and testing discipline will carry you into silicon. Start today with nandgame and HDLBits, and in a year you could be holding a chip with your own design on it.